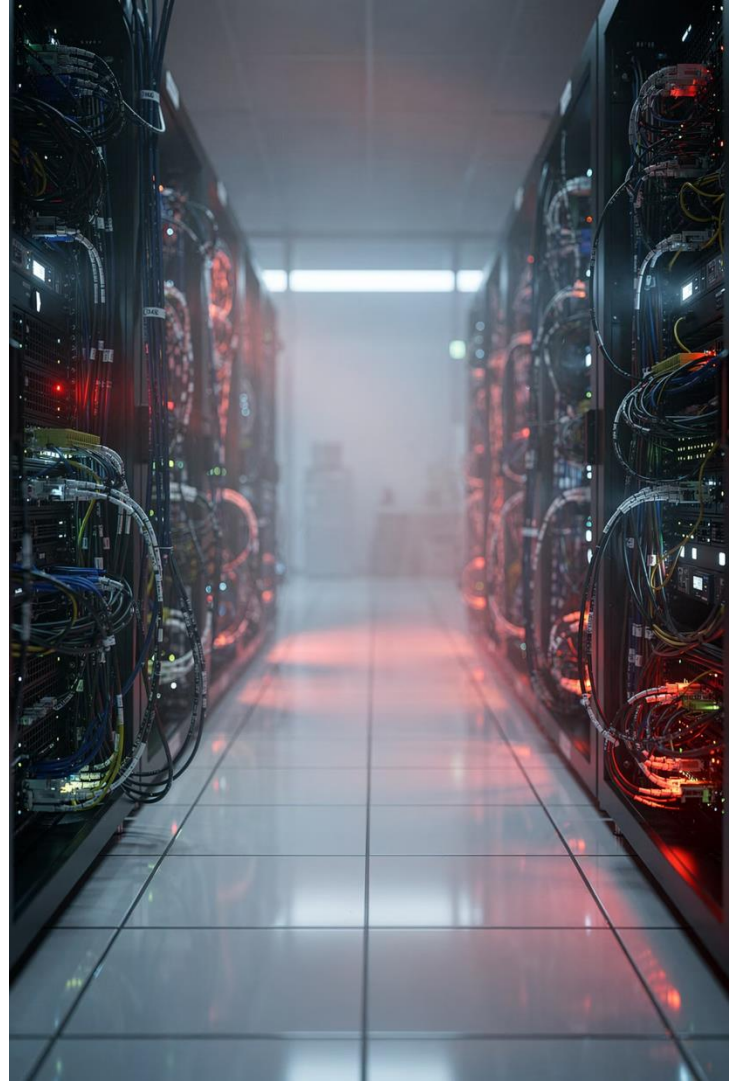


Sesión 4: Vulnerabilidades y Desarrollo Seguro

Módulo IV · Seguridad Aplicada a Entornos Java & .NET

3 H 20 MIN

NIVEL SENIOR



Estructura de la sesión

¿Qué vamos a cubrir hoy?

01

Principales vulnerabilidades backend

SQL Injection, XSS, CSRF y XXE: anatomía, impacto y ejemplos reales en Java y .NET.

02

Técnicas de mitigación

Validación de entradas, configuración segura de frameworks y principio de mínimo privilegio.

03

Taller práctico

Análisis de código vulnerable, aplicación de medidas correctoras y revisión de patrones inseguros frecuentes.

Contexto

¿Por qué importa la seguridad en el backend?

El backend es el perímetro real

La mayor parte de la lógica de negocio, el acceso a datos y la gestión de identidades reside en el backend. Un fallo aquí puede comprometer la totalidad del sistema, independientemente de cuántas capas de frontend existan.

Las vulnerabilidades más explotadas en producción siguen siendo, año tras año, las mismas que llevan décadas documentadas en el OWASP Top 10.

Coste real de una brecha

- Exposición de datos sensibles de clientes
- Daño reputacional irreversible
- Multas regulatorias (GDPR, PCI-DSS)
- Interrupción del servicio y pérdida de ingresos
- Coste medio de una brecha: >4 M\$ (IBM, 2023)



Principales Vulnerabilidades en Aplicaciones Backend

Parte 1 de 3 · Anatomía, ejemplos y vectores de ataque

Vulnerabilidad #1

SQL Injection

SQL Injection sigue siendo una de las vulnerabilidades más críticas y prevalentes. Ocurre cuando la aplicación construye consultas SQL concatenando directamente entradas del usuario sin validación ni parametrización, permitiendo al atacante alterar la lógica de la consulta.

¿Qué puede hacer un atacante?

- Extraer la totalidad de la base de datos
- Saltarse mecanismos de autenticación
- Modificar o eliminar registros críticos
- Ejecutar comandos en el sistema operativo (stacked queries)

Variantes principales

- **Classic SQLi:** respuesta directa en la aplicación
- **Blind SQLi:** inferencia por comportamiento (booleano/tiempo)
- **Out-of-band SQLi:** exfiltración por canal alternativo (DNS)

Ejemplo real en Java

El siguiente patrón es **extremadamente común** en bases de código legadas y es el primer vector que explotará cualquier escáner automatizado:

```
// Peligroso: concatena entrada del usuario directamente en la consulta SQL.
// Un atacante puede alterar la lógica de la query e incluso acceder a datos no autorizados.
public User findUserByEmail(String email) throws SQLException {
    String sql = "SELECT id, email, name FROM users WHERE email = '" + email + "'";

    try (Connection connection = dataSource.getConnection();
        Statement statement = connection.createStatement();
        ResultSet resultSet = statement.executeQuery(sql)) {

        if (resultSet.next()) {
            return new User(
                resultSet.getLong("id"),
                resultSet.getString("email"),
                resultSet.getString("name")
            );
        }
        return null;
    }
}
```

El mismo problema en C#

```
using System.Data.SqlClient;
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
public class UsersController : ControllerBase
{
    private readonly string _connectionString = "Server=.;Database=AppDb;Trusted_Connection=True;";

    [HttpGet("search")]
    public IActionResult SearchUsers(string username)
    {
        // VULNERABLE: user input is concatenated directly into the SQL query.
        // An attacker can inject SQL by controlling the "username" parameter.
        string query = "SELECT Id, Username, Email FROM Users WHERE Username = '" + username + "'";

        using (var connection = new SqlConnection(_connectionString))
        using (var command = new SqlCommand(query, connection))
        {
            connection.Open();

            using (var reader = command.ExecuteReader())
            {
                // Return matching users
                return Ok("Query executed successfully.");
            }
        }
    }
}

// Example of malicious input:
// username = ' OR 1=1--
// This could turn the WHERE clause into a condition that returns all users.
```

Consultas parametrizadas y ORM

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class ProductSearchService {
    private static final String URL = "jdbc:mysql://localhost:3306/appdb";
    private static final String USER = "app_user";
    private static final String PASSWORD = "secret";

    public void searchProducts(String searchTerm) {
        // SECURE: Use a parameterized query instead of string concatenation.
        // The database treats the input as data, not as part of the SQL command.
        String sql = "SELECT id, name, price FROM products WHERE name LIKE ?";

        try (Connection connection = DriverManager.getConnection(URL, USER, PASSWORD);
            PreparedStatement statement = connection.prepareStatement(sql)) {

            // Bind user input safely.
            statement.setString(1, "%" + searchTerm + "%");

            try (ResultSet rs = statement.executeQuery()) {
                while (rs.next()) {
                    long id = rs.getLong("id");
                    String name = rs.getString("name");
                    double price = rs.getDouble("price");

                    System.out.println(id + " | " + name + " | " + price);
                }
            }
        } catch (SQLException e) {
            // Handle the error without exposing SQL details to the user.
            e.printStackTrace();
        }
    }
}
```


SQL Injection · ORM

Hibernate y Entity Framework como capa de defensa

El uso de un ORM elimina la mayoría de los vectores de SQLi al abstraer la construcción de queries. Sin embargo, los ORMs no son inmunes si se usa SQL nativo o HQL construido con concatenación.

Java — Hibernate / JPA

Usa `TypedQuery` con parámetros nombrados. Evita `createNativeQuery` con strings dinámicos.

.NET — Entity Framework

LINQ genera SQL parametrizado por defecto. Usa `FromSqlInterpolated` en lugar de `FromSqlRaw` con concatenación.

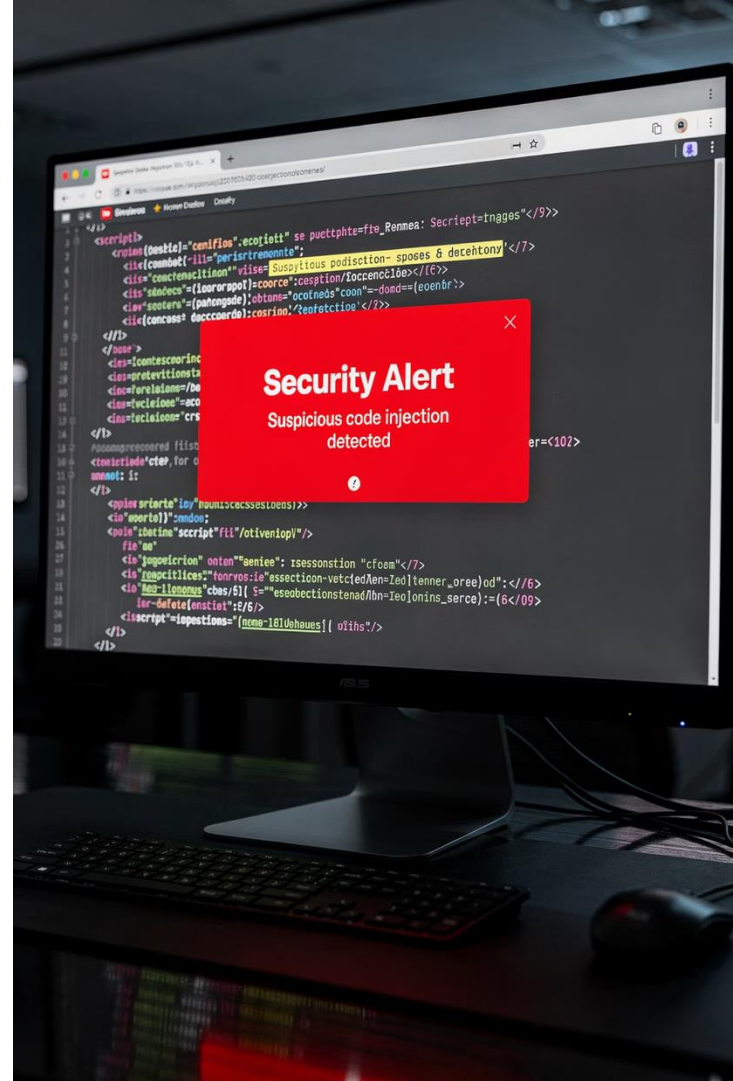
Regla de oro

Si necesitas SQL nativo, usa siempre parámetros nombrados. Jamás construyas la query interpolando variables de usuario.

Vulnerabilidad #2

Cross-Site Scripting (XSS)

XSS permite a un atacante inyectar scripts maliciosos que se ejecutan en el navegador de otros usuarios. Aunque se considera una vulnerabilidad de capa de presentación, su origen frecuente es el backend: APIs que devuelven datos sin escapar o plantillas renderizadas en el servidor.



Las tres variantes de XSS

1

Reflected XSS

El payload viaja en la petición (URL, formulario) y se refleja inmediatamente en la respuesta. Requiere que la víctima haga clic en un enlace malicioso. Ejemplo: parámetros de búsqueda mostrados sin escapar en la página de resultados.

2

Stored XSS

El payload se persiste en la base de datos (comentarios, perfiles, mensajes) y se ejecuta cada vez que otro usuario accede a esa página. Es el tipo más peligroso por su alcance masivo.

3

DOM-based XSS

La manipulación ocurre íntegramente en el cliente. El backend devuelve una respuesta legítima, pero el JavaScript del frontend escribe datos no saneados en el DOM. El servidor no tiene visibilidad directa.

Ejemplo práctico en Java (Spring MVC)

```
// VULNERABLE: Thymeleaf/JSP rendering user input without escaping
// If the user submits: <script>alert('XSS')</script>
// it will be executed in the browser.

@Controller
@RequestMapping("/comments")
public class CommentController {

    @GetMapping
    public String showForm(Model model) {
        model.addAttribute("comment", "");
        return "comment-form";
    }

    @PostMapping
    public String submitComment(@RequestParam String comment, Model model) {
        model.addAttribute("comment", comment);
        return "comment-form";
    }
}
```

```
<!-- comment-form.html (VULNERABLE) -->
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<body>
    <h2>Your comment</h2>

    <!-- th:text renders HTML unescaped -->
    <div th:text="${comment}"></div>

    <!-- JSP equivalent of unsafe raw output -->
    <%- ${comment} or <c:out escapeXml="false" --%>
</body>
</html>
```

```
// SECURE: escape the output before rendering
// Prefer th:text for automatic escaping, or HtmlUtils.htmlEscape() when needed.

@Controller
@RequestMapping("/secure-comments")
public class SecureCommentController {

    @GetMapping
    public String showSecureForm(Model model) {
        model.addAttribute("comment", "");
        return "secure-comment-form";
    }

    @PostMapping
    public String submitSecureComment(@RequestParam String comment, Model model) {
        model.addAttribute("comment", HtmlUtils.htmlEscape(comment));
        return "secure-comment-form";
    }
}
```

```
<!-- secure-comment-form.html (SECURE) -->
```

Razor Pages y ASP.NET Core

```
// Pages/Comment.cshtml
@page
@model CommentModel
```

Vulnerable example

```
@Html.Raw(Model.UserComment)
```

Secure example

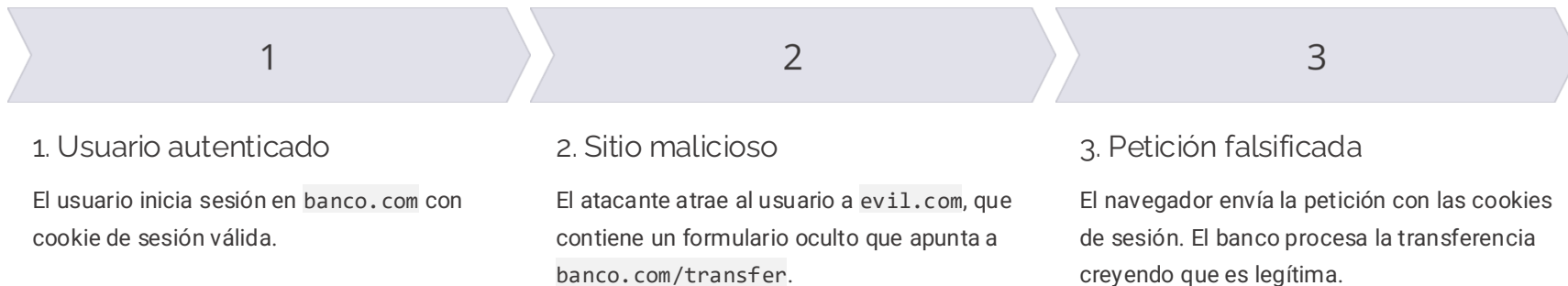
```
@Model.UserComment
```

```
// Pages/Comment.cshtml.cs using System.Text.Encodings.Web; using Microsoft.AspNetCore.Mvc.RazorPages; public class
CommentModel : PageModel { public string UserComment { get; set; } = ""; // Always prefer letting Razor encode values
automatically. public string EncodedComment { get; private set; } = ""; public void OnGet() { // Example of
encoding explicitly in code-behind when you need an encoded string // for display or storage in a safe, HTML-encoded
form. UserComment = Request.Query["comment"].ToString(); EncodedComment =
HtmlEncoder.Default.Encode(UserComment); } }
```

Vulnerabilidad #3

Cross-Site Request Forgery (CSRF)

CSRF engaña al navegador de un usuario autenticado para que envíe una petición no intencionada a una aplicación en la que tiene sesión activa. El servidor ve una petición aparentemente legítima porque incluye las cookies de sesión, pero la acción la inició un sitio malicioso.



Tokens CSRF y SameSite

```
// Spring Security – CSRF habilitado por defecto en aplicaciones web
@Configuration
public class SecurityConfig extends WebSecurityConfigurerAdapter {
    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf
                .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())
            );
        // Para APIs REST stateless, deshabilitar CSRF es aceptable
        // si se usa autenticación por token (JWT en cabecera Authorization)
        // .csrf(AbstractHttpConfigurer::disable)
    }
}
```

Token sincronizador

El servidor genera un token aleatorio único por sesión. El cliente debe incluirlo en cada petición de mutación. El sitio malicioso no puede leer este token por la política SOP.

SameSite Cookie

`SameSite=Strict` o `Lax` impide que el navegador envíe cookies en peticiones cross-site, eliminando el vector de ataque directamente a nivel de cookie.

Protección CSRF en el ecosistema .NET

```
// Program.cs – AntiForgery habilitado por defecto en Razor Pages
builder.Services.AddAntiforgery(options => {
    options.HeaderName = "X-CSRF-TOKEN"; // Para peticiones AJAX
    options.Cookie.SameSite = SameSiteMode.Strict;
    options.Cookie.SecurePolicy = CookieSecurePolicy.Always;
});

// En un controlador MVC
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Transfer(TransferViewModel model) { ... }
```

// Para Minimal APIs / JWT – desactivar AntiForgery es seguro
// si TODAS las peticiones de mutación requieren Authorization header

 Para APIs REST puras que usan JWT en cabecera **Authorization**, CSRF no aplica porque el atacante no puede hacer que el navegador incluya ese header de forma involuntaria.



Cómo funciona una entidad externa maliciosa

```
<!-- Payload XXE clásico – lectura de archivo del sistema -->
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [
  <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<userProfile>
  <username>&xxe;</username>
</userProfile>

<!-- Variante SSRF – acceso a metadatos de instancia en AWS -->
<!DOCTYPE foo [
  <!ENTITY xxe SYSTEM "http://169.254.169.254/latest/meta-data/iam/security-credentials/">
]>
```

- ❌ Si el parser devuelve el contenido del campo `username` en la respuesta, el atacante recibe el contenido del archivo o la respuesta HTTP interna directamente.

Deshabilitar DTDs y entidades externas

```
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.SAXParserFactory;
import javax.xml.parsers.ParserConfigurationException;
import org.xml.sax.SAXException;

public class XxeMitigationExample {

    public static DocumentBuilderFactory secureDocumentBuilderFactory() throws ParserConfigurationException {
        DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();

        // Bloquea completamente los DOCTYPE
        dbf.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);

        // Deshabilita entidades externas generales y de parámetro
        dbf.setFeature("http://xml.org/sax/features/external-general-entities", false);
        dbf.setFeature("http://xml.org/sax/features/external-parameter-entities", false);

        // Evita expansión de referencias a entidades
        dbf.setExpandEntityReferences(false);

        // Opcional: ignora DTDs externas
        dbf.setIncludeAware(false);
        dbf.setNamespaceAware(true);

        return dbf;
    }

    public static SAXParserFactory secureSAXParserFactory()
        throws ParserConfigurationException, SAXException {
        SAXParserFactory spf = SAXParserFactory.newInstance();

        // Bloquea completamente los DOCTYPE
        spf.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);

        // Deshabilita entidades externas generales y de parámetro
        spf.setFeature("http://xml.org/sax/features/external-general-entities", false);
        spf.setFeature("http://xml.org/sax/features/external-parameter-entities", false);

        // Buenas prácticas adicionales
        spf.setIncludeAware(false);
        spf.setNamespaceAware(true);

        return spf;
    }
}
```

XmlReaderSettings en C#

```
using System;
using System.Xml;

var settings = new XmlReaderSettings
{
    // Impide que el lector procese DTDs, bloqueando entidades externas.
    DtdProcessing = DtdProcessing.Prohibit,

    // Evita cualquier resolución externa de recursos XML.
    XmlResolver = null
};

using var reader = XmlReader.Create("input.xml", settings);
while (reader.Read())
{
    // Procesa el XML de forma segura.
}
```

Resumen comparativo

Las 4 vulnerabilidades a golpe de vista

Vulnerabilidad	Vector	Impacto	Mitigación clave
SQL Injection	Entradas HTTP → BD	Exfiltración / modificación de datos, bypass auth	Consultas parametrizadas, ORM
XSS	Datos persistidos / reflejados → navegador	Robo de sesión, phishing, defacement	Output encoding, CSP
CSRF	Petición cross-site con cookies	Acciones no autorizadas en nombre del usuario	Token CSRF, SameSite cookie
XXE	Documento XML malicioso → parser	Lectura de ficheros, SSRF, DoS	Deshabilitar DTDs y entidades externas



Técnicas de Mitigación

Parte 2 de 3 · Estrategias transversales de defensa en profundidad

Mitigación #1

Validación y saneamiento de entradas

La validación de entradas es la primera línea de defensa. Toda entrada que llegue desde el exterior —parámetros HTTP, cabeceras, cuerpos JSON/XML, cookies— debe considerarse no confiable hasta que se valide explícitamente contra un esquema esperado.

Validación (rechazar lo malo)

- Define un esquema de tipos, longitudes y formatos esperados
- Usa **allowlists** (listas blancas), nunca blocklists
- Valida en la capa de entrada: controlador, DTO, anotaciones
- Falla rápido: rechaza y responde 400 antes de procesar

Saneamiento (neutralizar lo peligroso)

- Escapa caracteres especiales según el contexto de uso (HTML, SQL, shell)
- Usa librerías especializadas: OWASP Java HTML Sanitizer, AntiXSS (.NET)
- No confundas saneamiento con validación: son complementarios
- Saneamiento ≠ solución para SQLi: usa parámetros igualmente

Bean Validation con Jakarta Validation API

```
// DTO con anotaciones de validación
public class CreateUserRequest {

    @NotBlank
    @Size(min = 3, max = 50)
    @Pattern(regexp = "^[a-zA-Z0-9_-]+$", message = "Solo caracteres alfanuméricos")
    private String username;

    @NotBlank
    @Email
    private String email;

    @NotNull
    @Min(18) @Max(120)
    private Integer age;
}

// Controlador Spring
@PostMapping("/users")
public ResponseEntity<Void> createUser(@Valid @RequestBody CreateUserRequest req) {
    // Si la validación falla, Spring lanza MethodArgumentNotValidException
    // y devuelve 400 automáticamente
    userService.create(req);
    return ResponseEntity.status(201).build();
}
```


Data Annotations y FluentValidation

```
// Data Annotations en el modelo
public class CreateUserRequest {
    [Required]
    [StringLength(50, MinimumLength = 3)]
    [RegularExpression(@"^[a-zA-Z0-9_-]+$")]
    public string Username { get; set; }

    [Required, EmailAddress]
    public string Email { get; set; }

    [Range(18, 120)]
    public int Age { get; set; }
}

// FluentValidation - más expresivo para reglas complejas
public class CreateUserValidator : AbstractValidator<CreateUserRequest> {
    public CreateUserValidator() {
        RuleFor(x => x.Username)
            .NotEmpty().Length(3, 50)
            .Matches(@"^[a-zA-Z0-9_-]+$");
        RuleFor(x => x.Email).NotEmpty().EmailAddress();
        RuleFor(x => x.Age).InclusiveBetween(18, 120);
    }
}
```

❗ **FluentValidation** permite separar las reglas de negocio del modelo y reutilizarlas fácilmente en tests unitarios independientes del framework web.

Mitigación #2

Configuración segura de frameworks

Los frameworks modernos como Spring Security y ASP.NET Core incluyen protecciones activadas por defecto, pero la configuración incorrecta o la desactivación negligente de estas medidas es una de las principales causas de vulnerabilidades en producción.



Security Headers

Configura `X-Content-Type-Options`, `X-Frame-Options`, `Strict-Transport-Security` y `Content-Security-Policy` en todas las respuestas.



Dependencias actualizadas

Mantén las versiones de Spring, .NET y sus dependencias al día. Activa `Dependabot` o `OWASP Dependency-Check` en el pipeline CI/CD.



Configuración por defecto

Nunca deshabilites protecciones del framework sin entender el impacto. Documenta toda excepción con su justificación y revisión periódica.



CORS restrictivo

Define explícitamente los orígenes permitidos. Evita `allowedOrigins("*")` en APIs con autenticación. Usa allowlists de dominios propios.

Headers de seguridad en Spring Boot

```
@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .headers(headers -> headers
                .contentTypeOptions(withDefaults())
                .frameOptions(frame -> frame.deny())
                .httpStrictTransportSecurity(hsts -> hsts
                    .includeSubDomains(true)
                    .maxAgeInSeconds(31536000)
                )
                .contentSecurityPolicy(csp -> csp
                    .policyDirectives("default-src 'self'; script-src 'self'; object-src 'none'")
                )
            )
            .csrf(csrf -> csrf.csrfTokenRepository(
                CookieCsrfTokenRepository.withHttpOnlyFalse())
            )
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
            );
        return http.build();
    }
}
```

Middleware de seguridad en .NET

```
// Program.cs
var app = builder.Build();

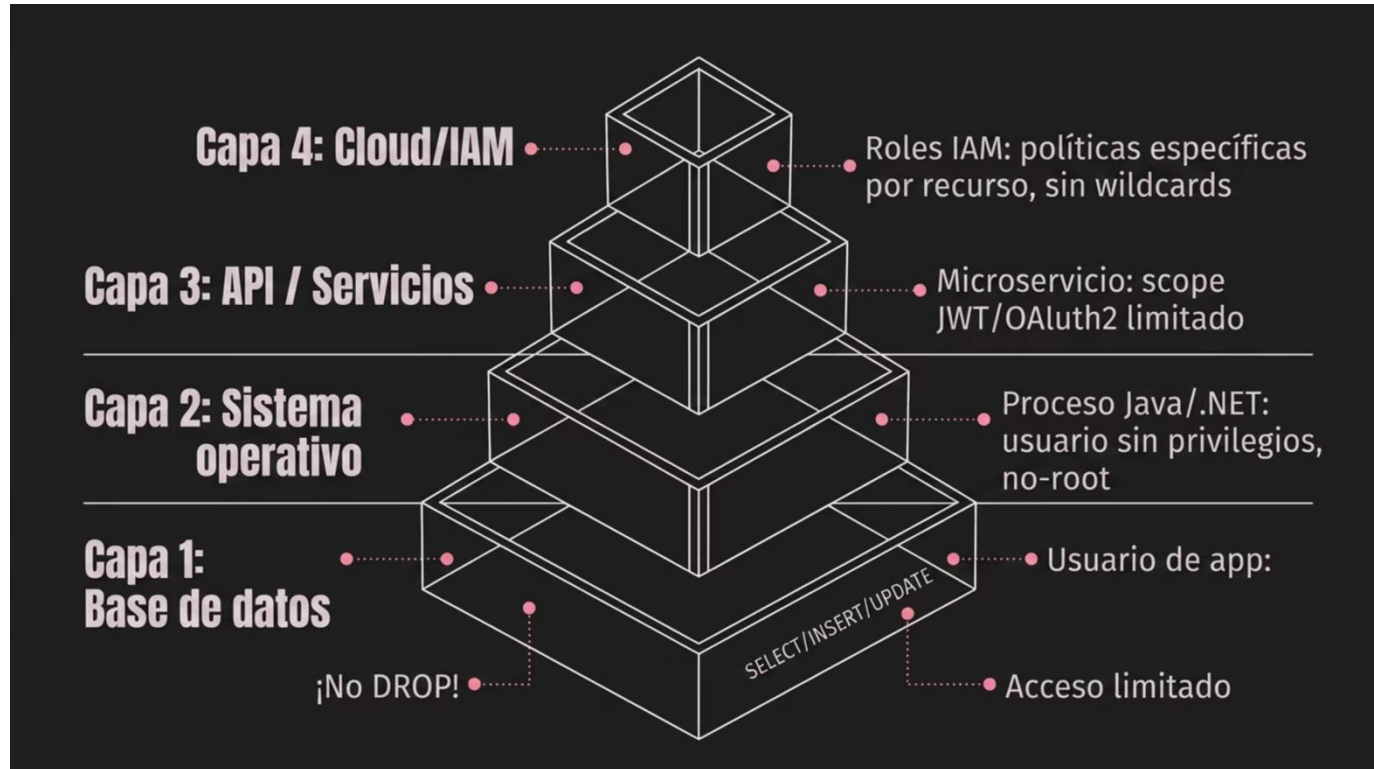
app.UseHsts(); // Strict-Transport-Security
app.UseHttpsRedirection();
app.UseXContentTypeOptions(); // NWebsec o middleware propio

app.Use(async (context, next) => {
    context.Response.Headers.Add("X-Frame-Options", "DENY");
    context.Response.Headers.Add("X-XSS-Protection", "1; mode=block");
    context.Response.Headers.Add("Referrer-Policy", "strict-origin-when-cross-origin");
    context.Response.Headers.Add(
        "Content-Security-Policy",
        "default-src 'self'; script-src 'self'; object-src 'none'"
    );
    await next();
});

app.UseCors(policy => policy
    .WithOrigins("https://myapp.com", "https://admin.myapp.com")
    .AllowedMethods("GET", "POST", "PUT", "DELETE")
    .AllowCredentials()
);
```

Principio de mínimo privilegio

El principio de mínimo privilegio (PoLP) establece que cada componente del sistema debe operar con los permisos estrictamente necesarios para realizar su función, y nada más. Aplicado consistentemente, limita drásticamente el radio de impacto de cualquier vulnerabilidad.



Usuarios de BD con permisos acotados

```
-- PostgreSQL: usuario restringido para la aplicación
CREATE USER app_user WITH PASSWORD 'cambia_esta_contraseña';

GRANT CONNECT ON DATABASE mi_base TO app_user;
GRANT USAGE ON SCHEMA public TO app_user;

-- Solo acceso a tablas específicas
GRANT SELECT, INSERT ON TABLE clientes, pedidos TO app_user;

-- Permisos por defecto más restrictivos para nuevas tablas
ALTER DEFAULT PRIVILEGES IN SCHEMA public
REVOKE ALL ON TABLES FROM app_user;

-- Quitar permisos peligrosos si existieran
REVOKE DROP, ALTER ON DATABASE mi_base FROM app_user;
REVOKE ALL PRIVILEGES ON DATABASE mi_base FROM app_user;

-- Lo que NO debes hacer:
-- Nunca uses el usuario root/sa para la aplicación.
-- Nunca concedas permisos de superusuario o acceso total a la BD.
```

Mínimo privilegio · OAuth2 Scopes

Ámbitos acotados en tokens JWT

Diseño de scopes granulares

En lugar de un token con acceso total, define scopes específicos por operación y recurso. Un microservicio de pagos no necesita leer el historial médico del usuario.

- `orders:read` — solo lectura de pedidos
- `orders:write` — creación y modificación
- `payments:process` — procesamiento de pagos
- `admin:users` — gestión de usuarios (solo backoffice)

Validación en Spring Security

```
http.authorizeHttpRequests(auth -> auth
    .requestMatchers(HttpMethod.GET, "/orders/**")
    .hasAuthority("SCOPE_orders:read")
    .requestMatchers(HttpMethod.POST, "/orders")
    .hasAuthority("SCOPE_orders:write")
    .requestMatchers("/admin/**")
    .hasAuthority("SCOPE_admin:users")
    .anyRequest().authenticated()
);
```



Taller Práctico

Parte 3 de 3 · Análisis de código vulnerable, corrección y patrones inseguros frecuentes

EJERCICIOS GUIADOS

JAVA & .NET

Metodología del taller

Cómo vamos a trabajar

01

Red Team: identificar

Revisar fragmentos de código real con vulnerabilidades intencionadas. Identificar el problema, clasificarlo (SQLi, XSS, CSRF...) y describir el vector de ataque y el impacto potencial.

02

Blue Team: corregir

Aplicar la medida correctora adecuada sobre el mismo fragmento. La solución debe ser equivalente funcionalmente pero segura, siguiendo las mejores prácticas del ecosistema (Spring / .NET).

03

Revisión colectiva

Puesta en común de soluciones, discusión de alternativas y análisis de los trade-offs entre seguridad y mantenibilidad del código. Se identifican los patrones inseguros recurrentes.

Análisis: endpoint de búsqueda vulnerable

Analiza el siguiente endpoint. Identifica **al menos dos vulnerabilidades** distintas antes de pasar a la siguiente diapositiva.

```
@RestController
@RequestMapping("/api/products")
public class ProductController {

    @Autowired
    private DataSource dataSource;

    @GetMapping("/search")
    public ResponseEntity<String> search(
        @RequestParam String keyword,
        @RequestParam String category) throws Exception {

        Connection conn = dataSource.getConnection();
        String sql = "SELECT id, name, description FROM products " +
            "WHERE name LIKE '%" + keyword + "%' " +
            "AND category = '" + category + "'";

        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(sql);

        StringBuilder html = new StringBuilder("<ul>");
        while (rs.next()) {
            html.append("<li>").append(rs.getString("name"))
                .append(": ").append(rs.getString("description"))
                .append("</li>");
        }
        html.append("</ul>");

        return ResponseEntity.ok(html.toString());
    }
}
```

Vulnerabilidades identificadas y corrección

Vulnerabilidad 1: SQL Injection

Los parámetros `keyword` y `category` se concatenan directamente en la query. Usar `PreparedStatement` con `?` y `setString()` para ambos parámetros.

Vulnerabilidad 2: XSS reflejado

El campo `description` de la BD se inserta como HTML crudo. Escapar con `HtmlUtils.htmlEscape(rs.getString("description"))` o, mejor, devolver JSON y delegar el renderizado al frontend.

Vulnerabilidad 3: gestión de recursos

La conexión no se cierra en un bloque `try-with-resources`, produciendo fugas de conexiones en producción. Usar `Spring JdbcTemplate` o `NamedParameterJdbcTemplate` que gestiona el ciclo de vida automáticamente.

Versión segura del endpoint

```
@RestController
@RequestMapping("/api/products")
public class ProductController {

    @Autowired
    private NamedParameterJdbcTemplate jdbcTemplate;

    @GetMapping("/search")
    public ResponseEntity<List<ProductDto>> search(
        @RequestParam @Size(max = 100) @Pattern(regexp = "^\\w\\s-$") String keyword,
        @RequestParam @NotBlank String category) {

        String sql = "SELECT id, name, description FROM products " +
            "WHERE name LIKE :keyword AND category = :category";

        MapSqlParameterSource params = new MapSqlParameterSource()
            .addValue("keyword", "%" + keyword + "%")
            .addValue("category", category);

        List<ProductDto> results = jdbcTemplate.query(sql, params,
            (rs, rowNum) -> new ProductDto(
                rs.getLong("id"),
                rs.getString("name"),
                rs.getString("description") // JSON serializado, no HTML
            )
        );

        return ResponseEntity.ok(results); // Jackson serializa a JSON seguro
    }
}
```

Análisis: controlador de perfil vulnerable

Identifica las vulnerabilidades en este controlador ASP.NET Core antes de ver la solución:

```
[ApiController]
[Route("api/[controller]")]
public class ProfileController : ControllerBase {

    private readonly IDbConnection _db;

    [HttpGet("{userId}")]
    public async Task<IActionResult> GetProfile(string userId) {
        // Sin verificar si el usuario autenticado puede ver este perfil
        var query = $"SELECT * FROM UserProfiles WHERE UserId = '{userId}'";
        var profile = await _db.QueryFirstOrDefaultAsync(query);
        return Ok(profile);
    }

    [HttpPut("{userId}")]
    public async Task<IActionResult> UpdateBio(string userId, [FromBody] string bio) {
        var query = $"UPDATE UserProfiles SET Bio = '{bio}' WHERE UserId = '{userId}'";
        await _db.ExecuteAsync(query);
        return NoContent();
    }
}
```

Tres problemas en ocho líneas de código

SQL Injection (x2)

Ambas queries usan interpolación de strings con Dapper. Solución: usar parámetros nombrados `@UserId` y `@Bio` con objeto anónimo en Dapper.

Broken Object Level Auth (BOLA)

El endpoint `GET /profile/{userId}` no verifica que el usuario autenticado tenga derecho a ver ese perfil. Cualquier usuario autenticado puede ver el perfil de cualquier otro. Comparar `userId` con el claim del JWT.

Missing Input Validation

El campo `bio` acepta cualquier contenido sin límite de longitud ni validación. Un payload de 10 MB puede causar DoS. Agregar `[MaxLength(500)]` y validación en el modelo.

Versión segura en ASP.NET Core

```
[HttpGet("{userId}")]
[Authorize]
public async Task<IActionResult> GetProfile(Guid userId) {
    // Verificar que el usuario autenticado solo accede a su propio perfil
    var currentUserId = Guid.Parse(User.FindFirstValue(ClaimTypes.NameIdentifier!));
    if (currentUserId != userId && !User.IsInRole("Admin"))
        return Forbid();

    var profile = await _db.QueryFirstOrDefaultAsync<UserProfile>(
        "SELECT * FROM UserProfiles WHERE UserId = @UserId",
        new { UserId = userId } // Parámetro nombrado – Dapper escapa automáticamente
    );
    return profile is null ? NotFound() : Ok(profile);
}

[HttpPut("{userId}")]
[Authorize]
public async Task<IActionResult> UpdateBio(Guid userId, [FromBody] UpdateBioRequest request) {
    var currentUserId = Guid.Parse(User.FindFirstValue(ClaimTypes.NameIdentifier!));
    if (currentUserId != userId) return Forbid();

    await _db.ExecuteAsync(
        "UPDATE UserProfiles SET Bio = @Bio WHERE UserId = @UserId",
        new { Bio = request.Bio, UserId = userId }
    );
    return NoContent();
}
```

Los errores que se repiten una y otra vez

1

Logging de datos sensibles

Loggear contraseñas, tokens JWT o PAN de tarjetas en claro en ficheros de log accesibles. Los logs deben sanitizarse antes de escribirse, eliminando o enmascarando datos sensibles.

2

Deserialización insegura

Deserializar objetos de fuentes no confiables sin validar el tipo. En Java, `ObjectInputStream` sin filtro de tipos puede llevar a RCE. Usa formatos como JSON con esquema explícito.

3

Exposición de stack traces

Devolver excepciones de Java/.NET completas en la respuesta HTTP en producción. Configura un `@ControllerAdvice` / `ExceptionHandler` global que devuelva mensajes genéricos.

4

Hardcoded credentials

Credenciales, API keys o connection strings embebidos directamente en el código fuente. Usa variables de entorno, Vault o los secret managers del proveedor cloud.

No loggees lo que no debes loggear

```
// BAD: Java SLF4J - logging passwords, tokens and credit cards in plain text
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class AuthService {
    private static final Logger log = LoggerFactory.getLogger(AuthService.class);

    public void login(String username, String password, String jwtToken, String creditCardNumber) {
        log.info("Login attempt username={}, password={}, token={}, card={}",
            username, password, jwtToken, creditCardNumber);
    }
}

// GOOD: Java SLF4J - mask/redact sensitive fields before logging
public class SafeAuthService {
    private static final Logger log = LoggerFactory.getLogger(SafeAuthService.class);

    public void login(String username, String password, String jwtToken, String creditCardNumber) {
        log.info("Login attempt username={}, password={}, token={}, card={}",
            username,
            "*****",
            redactToken(jwtToken),
            maskCard(creditCardNumber));
    }

    private String redactToken(String token) {
        if (token == null || token.length() <= 8) return "REDACTED";
        return token.substring(0, 4) + "... " + token.substring(token.length() - 4);
    }

    private String maskCard(String card) {
        if (card == null || card.length() < 4) return "REDACTED";
        return "**** * " + card.substring(card.length() - 4);
    }
}

// BAD: .NET - logging full stack traces and sensitive data to the user
public IActionResult Checkout(CheckoutRequest request)
{
    try
    {
        _paymentService.Charge(request.CardNumber, request.Cvv, request.CustomerToken);
        return Ok();
    }
    catch (Exception ex)
    {
        return Content($"Error: {ex}\nCard={request.CardNumber}\nToken={request.CustomerToken}");
    }
}

// GOOD: .NET - log a safe error ID instead of exposing details
public IActionResult Checkout(CheckoutRequest request)
{
    var errorId = Guid.NewGuid().ToString("N");

    try
    {
        _paymentService.Charge(request.CardNumber, request.Cvv, request.CustomerToken);
        return Ok();
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Checkout failed. ErrorId={ErrorId}", errorId);
        return StatusCode(500, new { message = "Something went wrong.", errorId });
    }
}
```

Deserialización insegura en Java

```
// BAD: Using ObjectInputStream.readObject() on untrusted data without validation
// Risk: remote code execution if a malicious serialized payload is provided
import java.io.ByteArrayInputStream;
import java.io.IOException;
import java.io.ObjectInputStream;

public class InsecureDeserializer {

    public Object deserialize(byte[] data) throws IOException, ClassNotFoundException {
        try (ObjectInputStream ois = new ObjectInputStream(new ByteArrayInputStream(data))) {
            return ois.readObject();
        }
    }
}

// GOOD: Use Jackson ObjectMapper with a specific known class instead of raw Java serialization
import com.fasterxml.jackson.databind.ObjectMapper;
import java.io.IOException;

public class SafeDeserializer {
    private final ObjectMapper objectMapper = new ObjectMapper();

    public User deserialize(byte[] data) throws IOException {
        return objectMapper.readValue(data, User.class);
    }
}

class User {
    public String username;
    public String email;
}
```

Gestión global de excepciones

```
// BAD Java: never catch Exception and return the raw message or stack trace to the client.
// This leaks internal details, class names, file paths, and implementation behavior.

@RestController
@RequestMapping("/api/orders")
class OrderController {

    @GetMapping("/{id}")
    public ResponseEntity getOrder(@PathVariable String id) {
        try {
            Order order = orderService.findById(id);
            return ResponseEntity.ok(order);
        } catch (Exception ex) {
            // BAD: exposing internal exception details to the caller
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
                .body(ex.getMessage());
        }
    }

    // Even worse: returning the full stack trace to the client
    catch (Exception ex) {
        StringWriter sw = new StringWriter();
        ex.printStackTrace(new PrintWriter(sw));

        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(sw.toString());
    }

    // GOOD Java: use centralized exception handling with @ControllerAdvice.
    // Log full details internally, but return only a generic error message to the client.

    @RestControllerAdvice
    public class GlobalExceptionHandler {

        private static final Logger log = LoggerFactory.getLogger(GlobalExceptionHandler.class);

        @ExceptionHandler(Exception.class)
        public ResponseEntity handleUnexpectedException(Exception ex, HttpServletRequest request) {
            // Log the full exception internally for debugging and incident response
            log.error("Unhandled exception while processing {} {}", request.getMethod(), request.getRequestURI(), ex);

            // Return a safe, generic response without stack trace or internal details
            ApiError error = new ApiError(
                "An unexpected error occurred.",
                "ERR_INTERNAL_SERVER_ERROR"
            );

            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(error);
        }

        public record ApiError(String message, String code) {}
    }

    // GOOD .NET: middleware that catches unhandled exceptions and returns ProblemDetails.
    // Do not expose ex.Message or stack traces to the client.

    public sealed class GlobalExceptionHandlerMiddleware
    {
        private readonly RequestDelegate _next;
        private readonly ILogger _logger;

        public GlobalExceptionHandlerMiddleware(
            RequestDelegate next,
            ILogger logger)
        {

```

Problem Details y middleware de errores

```
// Program.cs – configuración recomendada para .NET 8+
builder.Services.AddProblemDetails(options => {
    options.CustomizeProblemDetails = ctx => {
        ctx.ProblemDetails.Extensions["traceId"] = Activity.Current?.Id;
        // NO incluir: exception message, stack trace, inner exceptions
    };
});

app.UseExceptionHandler(); // Captura todas las excepciones no controladas
app.UseStatusCodePages(); // Respuestas estándar para 4xx sin body

// Diferencia entre Development y Production
if (app.Environment.IsDevelopment()) {
    app.UseDeveloperExceptionPage(); // Stack trace SOLO en desarrollo local
} else {
    app.UseExceptionHandler("/error"); // Respuesta genérica en producción
}
```



Problem Details (RFC 9457) es el estándar para respuestas de error en APIs HTTP. ASP.NET Core 8 lo adopta por defecto. Proporciona un formato consistente sin exponer información interna.

Ejercicio integrador: auditoría de un endpoint completo

Revisa el siguiente escenario y enumera **todas las medidas de seguridad** que implementarías antes de poner este endpoint en producción:

Escenario

Endpoint POST `/api/reports/generate` que recibe un XML con parámetros de filtrado, ejecuta una query en BD, genera un PDF y lo devuelve. Es accesible desde internet para clientes B2B con API key.

Vectores a evaluar

- Parsing del XML de entrada
- Construcción de la query de BD
- Autenticación y autorización del cliente
- Generación y entrega del PDF
- Rate limiting y abuso de la API
- Logging del acceso y la operación

Criterios de corrección

- XXE: deshabilitar DTDs en el parser XML
- SQLi: ORM o PreparedStatement con parámetros
- Auth: validar API key + JWT con scope `reports:generate`
- PDF: evitar path traversal en el nombre del fichero
- Rate limiting: máximo N peticiones/minuto por cliente
- Audit log: usuario, timestamp, parámetros (sin datos sensibles)

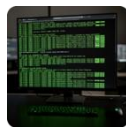
Herramientas del ciclo de desarrollo

Seguridad integrada en el pipeline



SAST — Análisis estático

Integra **SonarQube**, **Semgrep** o **Checkmarx** en el CI. Detectan SQLi, XSS y patrones inseguros en el código fuente antes del merge. Configura quality gates que bloqueen PRs con vulnerabilidades críticas.



SCA — Dependencias

OWASP Dependency-Check, **Snyk** o **Dependabot** analizan el árbol de dependencias contra bases de CVEs. Configura alertas automáticas para dependencias con CVSS ≥ 7 en el pipeline.

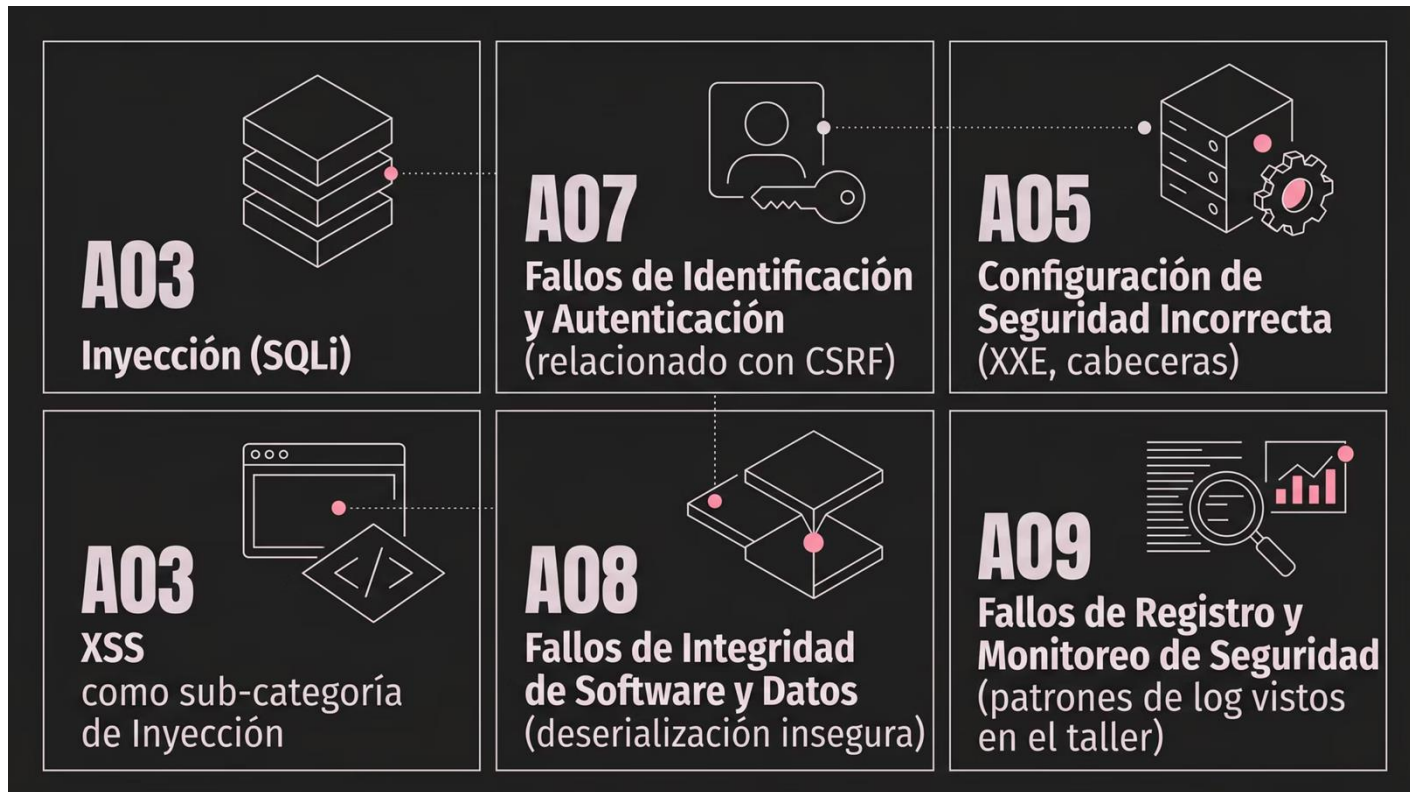


DAST — Análisis dinámico

OWASP ZAP o **Burp Suite Enterprise** atacan la aplicación en ejecución (entorno de staging) y detectan vulnerabilidades que el análisis estático no puede ver, como lógica de negocio incorrecta o misconfiguraciones.

Las vulnerabilidades vistas en contexto

Las cuatro vulnerabilidades de esta sesión cubren directamente **seis categorías del OWASP Top 10 (2021)**, que es el estándar de referencia de la industria para seguridad en aplicaciones web:



Buenas prácticas transversales

Secure by Design: principios de arquitectura



Defense in depth

Nunca confíes en una sola capa de seguridad. Apila validación de entrada, ORM, CSP, WAF y monitorización como líneas independientes de defensa.



Shift left security

La seguridad es más barata cuando se detecta en desarrollo que en producción. Integra SAST, revisiones de código enfocadas en seguridad y threat modeling desde el inicio del ciclo.



Fail securely

Cuando algo falla, el sistema debe quedar en un estado seguro: denegar el acceso por defecto, no exponer información interna y registrar el incidente.



Separation of concerns

Separa identidades, roles y responsabilidades: usuario de aplicación vs. usuario de migración, microservicio A vs. microservicio B, admin vs. usuario final.

Checklist de cierre

¿Qué debería hacer en mi próximo code review?



Busca activamente estos patrones

- ☐ Queries SQL construidas con concatenación o interpolación de strings
- ☐ Uso de `Statement` en lugar de `PreparedStatement`
- ☐ XML parseado sin deshabilitar DTDs ni entidades externas
- ☐ Outputs de BD o usuario insertados como HTML crudo
- ☐ Endpoints de mutación sin token CSRF o sin `SameSite` en cookies
- ☐ Excepciones con stack trace devueltas al cliente
- ☐ Credenciales o tokens hardcodeados en el código o en tests



Verifica que existen estas protecciones

- ☐ Anotaciones de validación (`@Valid`, Data Annotations) en todos los DTOs
- ☐ Security headers configurados en Spring Security / ASP.NET middleware
- ☐ Usuario de BD con permisos mínimos documentados
- ☐ Scopes OAuth2 granulares validados en cada endpoint protegido
- ☐ Logs sin datos sensibles en claro (contraseñas, tokens, PAN)
- ☐ Dependencias analizadas con SCA en el pipeline CI/CD

Conclusiones y Próximos Pasos

Lo que hemos cubierto

Anatomía y explotación de SQLi, XSS, CSRF y XXE con ejemplos reales en Java y .NET, mitigaciones concretas y análisis de código vulnerable en el taller.

Tarea para casa

Revisa un endpoint de tu proyecto actual con la checklist de esta sesión. Documenta los hallazgos y aplica al menos una mejora de las vistas hoy antes de la próxima sesión.

Próxima sesión

Auditoría y logging de acceso · Simulación de ataques con OWASP ZAP · Análisis de vulnerabilidades en dependencias con Dependency-Check.

Módulo IV · Seguridad Aplicada a Entornos Java & .NET · Sesión 4 de 4

